

MSYNC: pseudosynchrony

I. S. Plank

2025-07-28

Contents

1	Introduction	1
1.1	Package versions	1
2	MEA data	1
3	Speech data	6
3.1	Individual level: turn-based pseudosynchronisation	6
3.2	Dyad level: timecourse pseudosynchrony based on CCF	10
4	OpenFace data	14
5	OpenPose data	16
5.1	Conversation task	16
5.2	Mirror game	18

1 Introduction

This R Markdown script compares pseudosynchrony to synchrony values for the MSYNC project. In the project, participant were having a conversation as well as playing the mirror game together. The data was preprocessed before being read into this script. We use two approaches described in Moulder et al. (2018, Psychol Methods) to test the following hypotheses: “Synchrony does not exist between sections of size m in these two time series” (section sliding) and, in the case of the MEA data of the conversation task, “There is no time dependency at all between these two time series” (data shuffling). We use the same settings as when computing synchrony to measure synchrony (windowed-lagged cross-correlation with the same timing settings).

1.1 Package versions

The following packages are used in this RMarkdown file:

```
## [1] "R version 4.5.1 (2025-06-13)"
## [1] "knitr version 1.50"
## [1] "tidyverse version 2.0.0"
## [1] "ggpubr version 0.6.1"
## [1] "BayesFactor version 0.9.12.4.7"
## [1] "effectsize version 1.0.1"
## [1] "gggrain version 0.0.4"
## [1] "rMEA version 1.2.2"
```

2 MEA data

```
if (!file.exists(file.path(dt.path, "preprocessedMEA", "df_psync_MEA.csv"))) {

  # create a dataframe
  df.psync = data.frame()
```

```

# check if CT has been run before
if (!file.exists(file.path(dt.path, "preprocessedMEA", "df_psync-CT_MEA.csv"))) {
  # start with the conversation task
  attach(file.path(dt.path, "preprocessedMEA", "MEA_CT.Rdata"), name = 'CT')
  for (meth in c("seg", "data")) {
    if (meth == "set") {
      n1 = 400 # increase for the subject-specific comparison
    } else {
      n1 = 100
    }
    df.mea = pseudosync(meth, mea.scaled,
                        sampRate = 120,
                        lagSec = 5,
                        winSec = 30,
                        incSec = 15,
                        n = n1)

    df.mea = df.mea %>%
      mutate(
        dyad = sprintf("MSYNC_%s", gsub("all_(.*)_0.*", "\\1", name)),
        task = "CT",
        method = meth
      ) %>%
      merge(., df.ccf.agg %>% ungroup() %>%
            filter(position == "B" & measure == "mean") %>%
            select(dyad, phase, MEA.sync))
    df.psync = rbind(df.psync, df.mea)
    write_csv(df.psync, file.path(dt.path, "preprocessedMEA", "df_psync-CT.csv"))
  }
  # detach the CT data
  detach('CT')
} else {
  df.psync = read_csv(file.path(dt.path, "preprocessedMEA", "df_psync-CT_MEA.csv"))
}

# check if CT has been run before
if (!file.exists(file.path(dt.path, "preprocessedMEA", "df_psync-MG_MEA.csv"))) {
  # at the mirror game
  attach(file.path(dt.path, "preprocessedMEA", "MEA_MG.Rdata"), name = 'MG')
  for (meth in c("seg")) { # , "data"
    df.mea = pseudosync(meth, mea.scaled,
                        sampRate = 120,
                        lagSec = 2,
                        winSec = 30,
                        incSec = 1,
                        n = n)

    df.mea = df.mea %>%
      mutate(
        dyad = sprintf("MSYNC_%s", gsub("all_(.*)_0.*", "\\1", name)),
        task = "MG",
        method = meth,
        phase = as.numeric(substr(name, nchar(name), nchar(name)))
      ) %>%
      merge(., df.ccf.agg %>% ungroup() %>%
            filter(position == "B" & measure == "mean") %>%
            select(dyad, phase, MEA.sync))
    df.psync = rbind(df.psync, df.mea)
    write_csv(df.psync, file.path(dt.path, "preprocessedMEA", "df_psync-MG_MEA.csv"))
  }
  # detach the MG data

```

```

    detach('MG')
  } else {
    df.mea = read_csv(file.path(dt.path, "preprocessedMEA", "df_psync-MG_MEA.csv"))
    df.psync = rbind(df.psync, df.mea)
  }

  # save the pseudosynchrony values
  write_csv(df.psync, file.path(dt.path, "preprocessedMEA", "df_psync_MEA.csv"))

} else {
  df.psync = read_csv(file.path(dt.path, "preprocessedMEA", "df_psync_MEA.csv"))
}

df.pagg = df.psync %>%
  group_by(dyad, task, method) %>%
  summarise(
    psync = mean(as.numeric(psync)),
    MEA.sync = mean(MEA.sync)
  ) %>%
  pivot_longer(cols = c(psync, MEA.sync)) %>%
  mutate_if(is.character, as.factor)

aov.CT.seg = anovaBF( value ~ name,
  df.pagg %>% filter(task == "CT" & method == "seg"))

aov.CT.seg@baysFactor

##           bf           error           time           code
## name -0.9228907 0.000152056 Mon Jul 28 09:11:50 2025 87a35955bbb
interpret_bf(aov.CT.seg@baysFactor$bf, log = T)

## [1] "anecdotal evidence against"
## (Rules: jeffreys1961)

aov.CT.data = anovaBF( value ~ name,
  df.pagg %>% filter(task == "CT" & method == "data"))

aov.CT.data@baysFactor

##           bf           error           time           code
## name 72.11793 1.599553e-37 Mon Jul 28 09:11:50 2025 87a310b954da
interpret_bf(aov.CT.data@baysFactor$bf, log = T)

## [1] "extreme evidence in favour of"
## (Rules: jeffreys1961)

p1a = df.pagg %>%
  filter(task == "CT" & method == "seg") %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'fix1',
    boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
    violin.args = list(color = "black", outlier.shape = NA, alpha = a),
    point.args = list(show_guide = FALSE, alpha = a)
  ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
    aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
    aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +

```

```

labs(title = "Conversation task: segments",
      x = "",
      y = "") +
theme_bw() +
theme(legend.position = "none",
      plot.title = element_text(hjust = 0.5),
      text = element_text(size = 12))

p1b = df.pagg %>%
  filter(task == "CT" & method == "data") %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'fix1',
            boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
            violin.args = list(color = "black", outlier.shape = NA, alpha = a),
            point.args = list(show_guide = FALSE, alpha = a)
            ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
              aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
              aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Conversation task: data shuffling",
        x = "",
        y = "") +
  theme_bw() +
  theme(legend.position = "none",
        plot.title = element_text(hjust = 0.5),
        text = element_text(size = 12))

aov.MG = anovaBF( value ~ name,
                  df.pagg %>% filter(task != "CT"))

aov.MG@bayesFactor

```

```

##           bf           error           time           code
## name 59.34833 2.481753e-31 Mon Jul 28 09:11:50 2025 87a37771bcc9

interpret_bf(sort(aov.MG@bayesFactor$bf, decreasing = T)[1], log = T)

```

```

## [1] "extreme evidence in favour of"
## (Rules: jeffreys1961)

```

```

p2 = df.pagg %>%
  filter(task != "CT") %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'fix1',
            boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
            violin.args = list(color = "black", outlier.shape = NA, alpha = a),
            point.args = list(show_guide = FALSE, alpha = a)
            ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
              aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
              aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Mirror game",
        x = "",

```

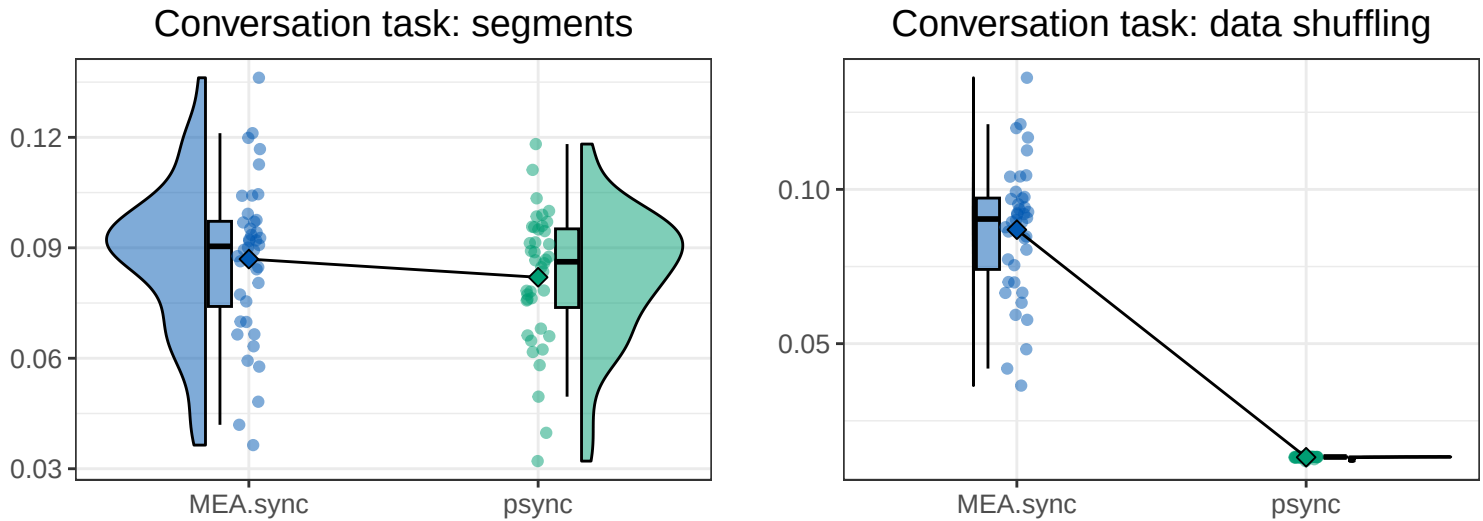
```

y = "" +
theme_bw() +
theme(legend.position = "none",
      plot.title = element_text(hjust = 0.5),
      text = element_text(size = 12))

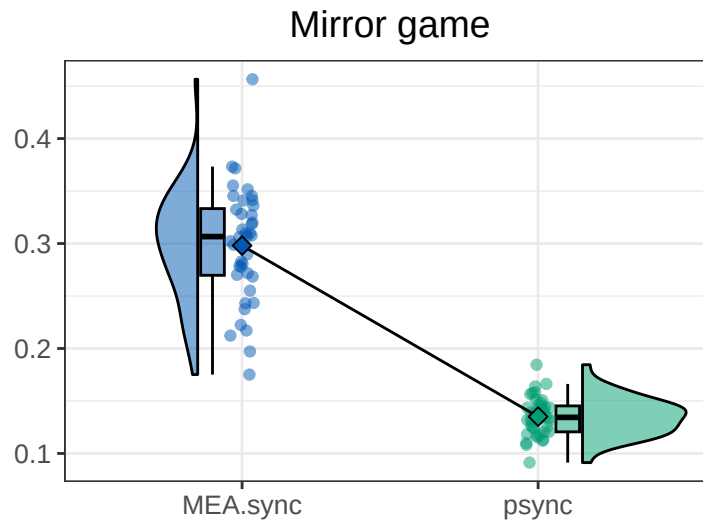
ggarrange(p1a, p1b, p2,
          ncol = 2)

```

```
## $`1`
```



```
##
## $`2`
```



```
##
## attr("class")
## [1] "list"      "ggarrange"
```

For the mirror game, synchrony credibly exceeds pseudosynchrony based on segments; thus, we do not compute pseudosynchrony based on data. For the conversation task, synchrony only exceeds pseudosynchrony based on data. Let's see whether we have at least some dyads that exceeded pseudosynchrony based on segments in the conversation task.

```

# initialise a dataframe
df.sub = data.frame()

# loop through dyads
for (d in unique(df.psync$dyad)) {

```

```

df.seg = df.psync %>% filter(method == "seg" & dyad == d & task == "CT")
t = ttestBF(x = df.seg$psync, mu = mean(df.seg$MEA.sync))
df.sub = rbind(df.sub,
  data.frame(
    dyad = d,
    bf.log = t@bayesFactor$bf,
    psync = mean(df.seg$psync),
    MEA.sync = mean(df.seg$MEA.sync)
  ))
}

df.sub = df.sub %>%
  mutate(
    direction = if_else(psync < MEA.sync,
      "less", "greater"),
    relevant = if_else(direction == "less" & bf.log >= log(3), "credible", "not credible"),
    interpretation = interpret_bf(bf.log, log = T)
  ) %>%
  # merge with movement quantity
  merge(., read_csv(file.path(dt.path, "MSYNC_mea_CT.csv"), show_col_types = F) %>%
    filter(position == "B") %>%
    select(dyad, total.mv) %>% distinct()) %>%
  mutate_if(is.character, as.factor) %>%
  write_csv(., file.path(dt.path, "preprocessedMEA", "df_psync_sub_MEA.csv"))

df.sub %>% group_by(relevant) %>% count()

## # A tibble: 2 x 2
## # Groups:   relevant [2]
##   relevant      n
##   <fct>      <int>
## 1 credible    26
## 2 not credible 14

# does it have anything to do with the overall movement quantity?
aov.CT.mv = anovaBF(total.mv ~ relevant, data = df.sub)

# nope, there is no difference between those
aov.CT.mv@bayesFactor

##           bf           error           time           code
## relevant -0.9375483 4.076742e-05 Mon Jul 28 09:11:51 2025 87a3254e19d8

```

About one third the dyads did not exceed pseudosynchrony when shuffling the segments. We might want to come back and investigate this later.

3 Speech data

3.1 Individual level: turn-based pseudosynchronisation

For turn-based pseudosynchronisation, this script uses an approach described in Moulder et al. (2018, *Psychol Methods*) to test the following hypothesis: “Synchrony does not exist between sections of size m in these two time series” (section sliding). This segment shuffling, from Moulder et al., “requires researchers to cut a time series X into shorter sections of size m which are randomly appended to one another to create a new time series X_s until no section is in its original position”. This new time series is then paired with an unchanged time series Y . Since the synchronisation is already based on sections or speech turns in the conversation, no data shuffling will be performed. Since each dyad can have a unique amount of turns, no dyad shuffling will be performed.

For each speaker in each task, the order of the previous turns are shuffled 100 times while the order of the current turns remain unchanged. Note that no check for uniqueness is performed. In the end, an average for this specific speaker and task is calculated from the 100 pseudosynchrony values.

```

filename = "indi_st-shuffle.RData"

if (!file.exists(filename)) {

  speaker = c()
  psyncint = c()
  psyncpit = c()
  psyncart = c()

  df = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_turns.csv"),
                show_col_types = F) %>%
    mutate(speaker = paste(name, speaker, sep = "_")) %>%
    select(c(speaker, pit_turn, pit_prev_turn,
             int_turn, int_prev_turn,
             art_prev_turn, art_turn))

  for (s in unique(df$speaker)) {
    speaker = c(speaker, rep(s, times = n))
    df.sel = df %>% filter(speaker == s)
    for (i in 1:n) {
      # shuffling
      df.sel$pit_prev = sample(df.sel$pit_prev_turn)
      df.sel$int_prev = sample(df.sel$int_prev_turn)
      df.sel$art_prev = sample(df.sel$art_prev_turn)
      psyncint = c(psyncint, abs(cor(df.sel$int_turn, df.sel$int_prev, use = "na.or.complete")))
      psyncpit = c(psyncpit, abs(cor(df.sel$pit_turn, df.sel$pit_prev, use = "na.or.complete")))
      psyncart = c(psyncart, abs(cor(df.sel$art_turn, df.sel$art_prev, use = "na.or.complete")))
    }
  }
  df.s_shuffle = data.frame(speaker, psyncint, psyncpit, psyncart)
  save(df.s_shuffle, file = file.path(dt.path, "preprocessedAudio", filename))
} else {
  load(filename)
}

df.sync = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_sync-indi.csv"),
                  show_col_types = F) %>%

  mutate(
    speaker = paste(name, speaker, sep = "_")
  ) %>%
  select(c(speaker, pit_sync, int_sync, art_sync))

df.agg = df.s_shuffle %>% group_by(speaker) %>%
  summarise(
    pit_psync = mean(psyncpit),
    int_psync = mean(psyncint),
    art_psync = mean(psyncart)
  )

df.agg = merge(df.sync, df.agg)

res = ttestBF(x = df.agg[!is.na(df.agg$pit_sync),]$pit_sync,
              y = df.agg[!is.na(df.agg$pit_sync),]$pit_psync,
              paired = T)

res

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 6.29357 ±0%
##

```

```

## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

## [1] 1.839528
interpret_bf(res@bayesFactor[["bf"]], log = T)

## [1] "moderate evidence in favour of"
## (Rules: jeffreys1961)
res = ttestBF(x = df.agg[!is.na(df.agg$int_sync),]$int_sync,
              y = df.agg[!is.na(df.agg$int_sync),]$int_psync,
              paired = T)
res

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 4117.749 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

## [1] 8.323062
interpret_bf(res@bayesFactor[["bf"]], log = T)

## [1] "extreme evidence in favour of"
## (Rules: jeffreys1961)
res = ttestBF(x = df.agg[!is.na(df.agg$art_sync),]$art_sync,
              y = df.agg[!is.na(df.agg$art_sync),]$art_psync,
              paired = T)
res

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 169.7583 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

## [1] 5.134375
interpret_bf(res@bayesFactor[["bf"]], log = T)

## [1] "extreme evidence in favour of"
## (Rules: jeffreys1961)
p1 = df.agg %>%
  pivot_longer(cols = c(pit_sync, pit_psync)) %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'fix1',
            boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
            violin.args = list(color = "black", outlier.shape = NA, alpha = a),
            point.args = list(show_guide = FALSE, alpha = a)
  )

```

```

    ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
    aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
    aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Turn-based pitch sync",
    x = "",
    y = "") +
  theme_bw() +
  theme(legend.position = "none",
    plot.title = element_text(hjust = 0.5),
    text = element_text(size = 12))

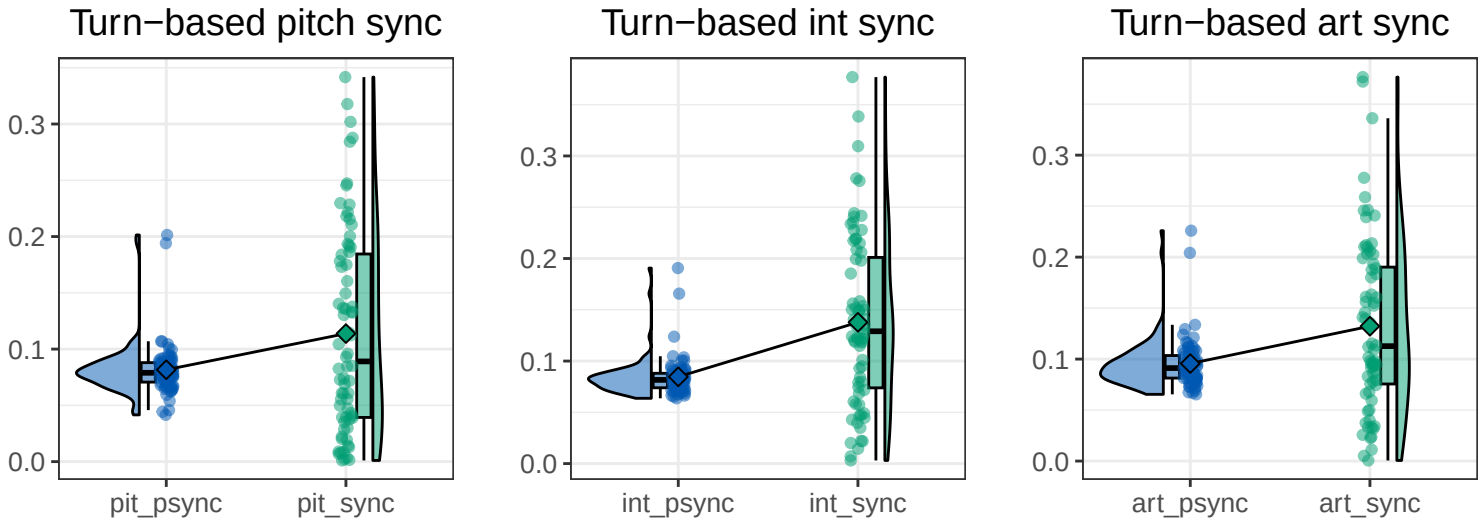
p2 = df.agg %>%
  pivot_longer(cols = c(int_sync, int_psync)) %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'fix1',
    boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
    violin.args = list(color = "black", outlier.shape = NA, alpha = a),
    point.args = list(show_guide = FALSE, alpha = a)
  ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
    aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
    aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Turn-based int sync",
    x = "",
    y = "") +
  theme_bw() +
  theme(legend.position = "none",
    plot.title = element_text(hjust = 0.5),
    text = element_text(size = 12))

p3 = df.agg %>%
  pivot_longer(cols = c(art_sync, art_psync)) %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'fix1',
    boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
    violin.args = list(color = "black", outlier.shape = NA, alpha = a),
    point.args = list(show_guide = FALSE, alpha = a)
  ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
    aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
    aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Turn-based art sync",
    x = "",
    y = "") +
  theme_bw() +
  theme(legend.position = "none",

```

```
plot.title = element_text(hjust = 0.5),
text = element_text(size = 12))
```

```
ggarrange(p1, p2, p3, ncol = 3)
```



The t-tests show that there is a significant difference between the pseudosynchronisation values obtained through segment shuffling and the real synchronisation values in the case of turn-based synchronisation of intensity and articulation rate, with the real synchronisation values being higher. However, for pitch this was not credibly the case.

3.2 Dyad level: timecourse pseudosynchrony based on CCF

We again use segment shuffling to compare pseudosynchrony and synchrony values on the dyad level. Pitch and intensity synchrony was calculated using the CCF function from rMEA. Therefore, we use the same function with the same window size but with shuffled windows for calculating pseudosynchrony.

For each speaker in each task, their time series is shuffled 50 times and then paired with the unchanged time series of their partner in the dyad. Note that no check for uniqueness is performed. In the end, an average for this specific dyad and task is calculated from the 100 pseudosynchrony values.

```
filename = "dyad_s-shuffle.RData"

if (!file.exists(file.path(dt.path, "preprocessedAudio", filename))) {

  # create new data frame for pseudosync values
  cols = c("dyad", "task", "pit_psync_MEA", "int_psync_MEA")
  df.psync = data.frame(matrix(nrow = 0, ncol = length(cols)))
  colnames(df.psync) = cols

  # load actual sync values
  df.dyad = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_dyad.csv"),
                     show_col_types = F)

  sr = 100

  # go through all dyads
  for (i in 1:nrow(df.dyad)) {
    # load the respective data of continuous pitch and intensity
    ptn = paste(df.dyad$dyad[i], df.dyad$task[i], ".*", "cont.csv", sep = "_")
    fls = list.files(pattern = ptn, path = file.path(dt.path, "preprocessedAudio"))
    df_L = read_csv(file.path(dt.path, "preprocessedAudio", fls[1]),
                    show_col_types = F, col_types = cols(.default = "d"))
    df_R = read_csv(file.path(dt.path, "preprocessedAudio", fls[2]),
                    show_col_types = F, col_types = cols(.default = "d"))
```

```

# resample to 100Hz
L_pit = df_L$pitch
L_pit[is.na(L_pit)] = 0
L_pit = signal::resample(L_pit,1,1000/sr)
L_int = df_L$int
L_int[is.na(L_int)] = 0
L_int = signal::resample(L_int,1,1000/sr)
R_pit = df_R$pitch
R_pit[is.na(R_pit)] = 0
R_pit = signal::resample(R_pit,1,1000/sr)
R_int = df_R$int
R_int[is.na(R_int)] = 0
R_int = signal::resample(R_int,1,1000/sr)
# create fakeMEA and divide into segments
mea_int = fakeMEA(L_int,R_int,sr,"int_L","int_R")
mea_pit = fakeMEA(L_pit,R_pit,sr,"pit_L","pit_R")
mea_int_L = split(mea_int[["all_01_01"]][["MEA"]][["s1Name"]],
                  floor(seq_along(mea_int[["all_01_01"]][["MEA"]][["s1Name"]])/16))
mea_int_R = split(mea_int[["all_01_01"]][["MEA"]][["s2Name"]],
                  floor(seq_along(mea_int[["all_01_01"]][["MEA"]][["s2Name"]])/16))
mea_pit_L = split(mea_pit[["all_01_01"]][["MEA"]][["s2Name"]],
                  floor(seq_along(mea_pit[["all_01_01"]][["MEA"]][["s1Name"]])/16))
mea_pit_R = split(mea_pit[["all_01_01"]][["MEA"]][["s2Name"]],
                  floor(seq_along(mea_pit[["all_01_01"]][["MEA"]][["s2Name"]])/16))
# shuffle each side n/2 times
for (j in 1:(n/2)) {

  # shuffling left
  mea_int[["all_01_01"]][["MEA"]][,2] = unlist(mea_int_R)
  mea_int[["all_01_01"]][["MEA"]][,1] = unlist(sample(mea_int_L))
  mea_int = MEAccf(mea_int,2,16,8)
  mea_pit[["all_01_01"]][["MEA"]][,2] = unlist(mea_pit_R)
  mea_pit[["all_01_01"]][["MEA"]][,1] = unlist(sample(mea_pit_L))
  mea_pit = MEAccf(mea_pit,2,16,8)
  # create new row with mean of peaks of windows
  new = c(df.dyad$dyad[i],
          df.dyad$task[i],
          mean(apply(mea_int$all_01_01$ccf,1,max)),
          mean(apply(mea_pit$all_01_01$ccf,1,max)))
  df.psync[nrow(df.psync) + 1,] = new

  # shuffling right
  mea_int[["all_01_01"]][["MEA"]][,2] = unlist(sample(mea_int_R))
  mea_int[["all_01_01"]][["MEA"]][,1] = unlist(mea_int_L)
  mea_int = MEAccf(mea_int,2,16,8)
  mea_pit[["all_01_01"]][["MEA"]][,2] = unlist(sample(mea_pit_R))
  mea_pit[["all_01_01"]][["MEA"]][,1] = unlist(mea_pit_L)
  mea_pit = MEAccf(mea_pit,2,16,8)
  # create new row with mean of peaks of windows
  new = c(df.dyad$dyad[i],
          df.dyad$task[i],
          mean(apply(mea_int$all_01_01$ccf,1,max)),
          mean(apply(mea_pit$all_01_01$ccf,1,max)))
  df.psync[nrow(df.psync) + 1,] = new
}
}
# save everything
save(df.psync, file = file.path(dt.path, "preprocessedAudio", filename))
} else {

```

```

load(file.path(dt.path, "preprocessedAudio", filename))
}

df.sync = read_csv(file.path(dt.path, "preprocessedAudio", "OUT_dyad.csv"),
                    show_col_types = F) %>%
select(c(dyad, task, pit_sync_MEA, int_sync_MEA))

df.agg = df.psync %>%
mutate(
  pit_psync_MEA = as.numeric(pit_psync_MEA),
  int_psync_MEA = as.numeric(int_psync_MEA)
) %>%
group_by(dyad, task) %>%
summarise(
  pit_psync_MEA = mean(pit_psync_MEA),
  int_psync_MEA = mean(int_psync_MEA)
)

df.agg = merge(df.sync, df.agg)

res = ttestBF(x = df.agg[!is.na(df.agg$pit_sync_MEA),]$pit_sync_MEA,
              y = df.agg[!is.na(df.agg$pit_sync_MEA),]$pit_psync_MEA,
              paired = T)
res

```

```

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 41.52436 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

```

```

## [1] 3.72628
interpret_bf(res@bayesFactor[["bf"]], log = T)

```

```

## [1] "very strong evidence in favour of"
## (Rules: jeffreys1961)

res = ttestBF(x = df.agg[!is.na(df.agg$int_sync_MEA),]$int_sync_MEA,
              y = df.agg[!is.na(df.agg$int_sync_MEA),]$int_psync_MEA,
              paired = T)
res

```

```

## Bayes factor analysis
## -----
## [1] Alt., r=0.707 : 2.496086e+22 ±0%
##
## Against denominator:
##   Null, mu = 0
## ---
## Bayes factor type: BFoneSample, JZS
res@bayesFactor[["bf"]]

```

```

## [1] 51.5716
interpret_bf(res@bayesFactor[["bf"]], log = T)

## [1] "extreme evidence in favour of"

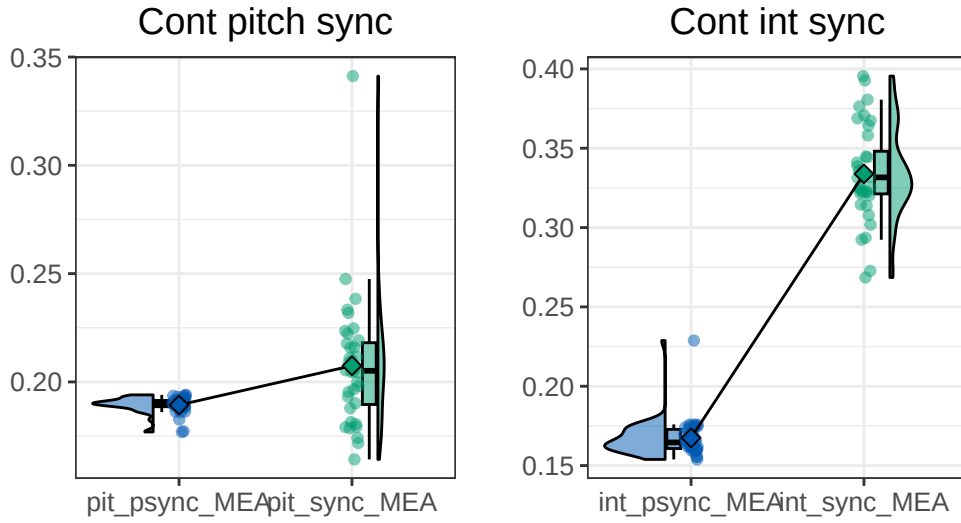
```

```
## (Rules: jeffreys1961)
```

```
p1 = df.agg %>%
  pivot_longer(cols = c(pit_sync_MEA, pit_psync_MEA)) %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'flx1',
    boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
    violin.args = list(color = "black", outlier.shape = NA, alpha = a),
    point.args = list(show_guide = FALSE, alpha = a)
  ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
    aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
    aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Cont pitch sync",
    x = "",
    y = "") +
  theme_bw() +
  theme(legend.position = "none",
    plot.title = element_text(hjust = 0.5),
    text = element_text(size = 12))

p2 = df.agg %>%
  pivot_longer(cols = c(int_sync_MEA, int_psync_MEA)) %>%
  ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
  geom_rain(rain.side = 'flx1',
    boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
    violin.args = list(color = "black", outlier.shape = NA, alpha = a),
    point.args = list(show_guide = FALSE, alpha = a)
  ) +
  stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
    aes(group = name)) +
  stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
    aes(group = name), colour = "black") +
  scale_fill_manual(values = custom.col[c(1,3)]) +
  scale_colour_manual(values = custom.col[c(1,3)]) +
  labs(title = "Cont int sync",
    x = "",
    y = "") +
  theme_bw() +
  theme(legend.position = "none",
    plot.title = element_text(hjust = 0.5),
    text = element_text(size = 12))

ggarrange(p1, p2, ncol = 3)
```



The t-tests show that there is a significant difference between the pseudosynchrony values obtained through segment shuffling and the real synchrony values in the case of MEA-based CCF synchrony of intensity and pitch, with the real synchrony values being higher.

4 OpenFace data

We only used separate cameras to record facial expressions in the conversation task.

```
if (!file.exists(file.path(dt.path, "preprocessedOF", "df_psync_OF.csv"))) {

  # create a dataframe
  df.psync = data.frame()

  # only do the conversation task
  attach(file.path(dt.path, "preprocessedOF", "OpenFace_CT.Rdata"), name = 'CT')
  for (meth in c("seg")) {
    df.mea = pseudosync(meth,
                        ls.fakeAU,
                        sampRate = 120,
                        lagSec = 2, winSec = 7, incSec = 4,
                        n = n)

    df.mea = df.mea %>%
      mutate(
        dyad = gsub("(.)_CT.*", "\\1", name),
        phase = "CT",
        method = meth,
        key = gsub(".*_CT_(.)", "\\1", name),
        psync = as.numeric(psync)
      ) %>%
      merge(., readRDS(file.path(dt.path, "MSYNC_AU_sync_CT.rds"))) %>%
      ungroup() %>%
      filter(position == "B" & measure == "mean") %>%
      select(dyad, phase, key, OF.sync))
    df.psync = rbind(df.psync, df.mea)
  }
  df.psync = rbind(df.psync, df.mea)
  # detach the CT data
  detach('CT')

  # save the pseudosynchrony values
  write_csv(df.psync, file.path(dt.path, "preprocessedOF", "df_psync_OF.csv"))

} else {
```

```

df.psync = read_csv(file.path(dt.path, "preprocessedOF", "df_psync_OF.csv"))
}

# initialise a dataframe
df.key = data.frame()

# loop through keys
for (d in unique(df.psync$key)) {
  df.seg = df.psync %>% filter(method == "seg" & key == d & phase == "CT") %>%
    group_by(dyad) %>%
    summarise(psync = mean(psync), OF.sync = mean(OF.sync))
  t = ttestBF(x = df.seg$psync, y = df.seg$OF.sync)
  df.key = rbind(df.key,
    data.frame(
      key = d,
      bf.log = t@bayesFactor$bf,
      psync = mean(df.seg$psync),
      OF.sync = mean(df.seg$OF.sync)
    ))
}

df.key = df.key %>%
  mutate(
    direction = if_else(psync < OF.sync,
      "less", "greater"),
    relevant = if_else(direction == "less" & bf.log >= log(3), "credible", "not credible"),
    interpretation = interpret_bf(bf.log, log = T)
  ) %>%
  mutate_if(is.character, as.factor) %>%
  write_csv(., file.path(dt.path, "preprocessedOF", "df_psync_key_OF.csv"))

# which keys can we use?
df.rel = df.key %>% filter(relevant == "credible")
kable(df.rel)

```

key	bf.log	psync	OF.sync	direction	relevant	interpretation
AU06_r	3.717618	0.2862024	0.3248038	less	credible	very strong evidence in favour of
AU12_r	2.258659	0.3128452	0.3430986	less	credible	moderate evidence in favour of

```

nkeys = length(unique(df.rel$key))
p = vector("list", nkeys)

for (k in 1:nkeys) {
  p[[k]] = df.psync %>%
    filter(key == unique(df.rel$key)[k]) %>%
    group_by(dyad) %>%
    summarise(
      psync = mean(psync),
      OF.sync = mean(OF.sync)
    ) %>%
    pivot_longer(cols = c(psync, OF.sync)) %>%
    mutate_if(is.character, as.factor) %>%
    ggplot(., aes(x = name, y = value, fill = name, colour = name)) + #
    geom_rain(rain.side = 'f1x1',
      boxplot.args = list(color = "black", outlier.shape = NA, show_guide = FALSE, alpha = a),
      violin.args = list(color = "black", outlier.shape = NA, alpha = a),
      point.args = list(show_guide = FALSE, alpha = a)
    ) +
    stat_summary(fun = mean, geom = "line", aes(group = 1), colour = "black") +

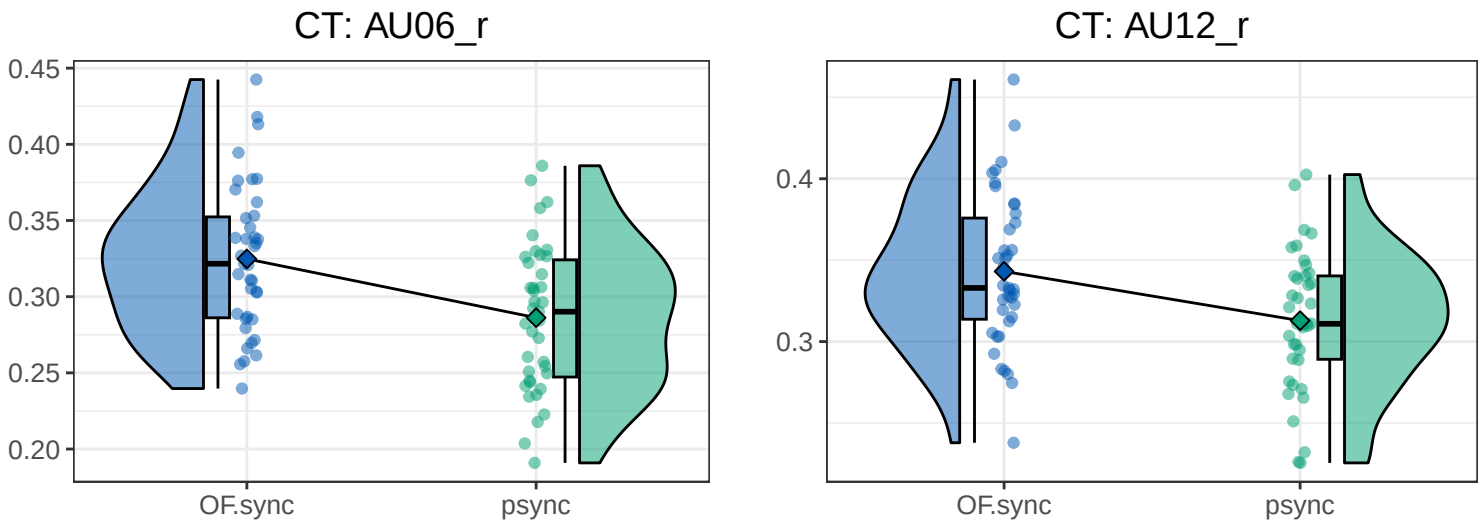
```

```

stat_summary(fun = mean, geom = "point", shape = 23, size = 2,
             aes(group = name)) +
stat_summary(fun = mean, geom = "point", shape = 5, size = 2,
             aes(group = name), colour = "black") +
scale_fill_manual(values = custom.col[c(1,3)]) +
scale_colour_manual(values = custom.col[c(1,3)]) +
labs(title = sprintf("CT: %s", unique(df.rel$key)[k]),
     x = "",
     y = "") +
theme_bw() +
theme(legend.position = "none",
      plot.title = element_text(hjust = 0.5),
      text = element_text(size = 12))
}

ggarrange(plotlist = p, ncol = 2)

```



5 OpenPose data

To compute IPS from the OpenPose data, we followed the postural mirroring approach from Fujiwara and colleagues (2019). However, they used an individual reference point for each interaction partner, specifically the neck. We chose the hip instead of the neck for the conversation task because it is more stable during seated conversations. For the mirror game, we did not use an individual reference point because people were moving freely with their whole body.

5.1 Conversation task

```

meth = "seg"
task = "CT"
filename = sprintf("df_psync_%s_OP-%s.csv", task, meth)
if (!file.exists(file.path(dt.path, "preprocessedOP", filename))) {

  # create a dataframe
  df.psync = data.frame()

  attach(file.path(dt.path, "preprocessedOP",
                  sprintf("MSYNC_OP_%s.Rdata", task)), name = task)
  df.meas = pseudosync(meth, meas.ccf,
                      sampRate = 120,
                      lagSec = 5, winSec = 30, incSec = 15,
                      n = n)
}

```

```

df.mea = df.mea %>%
  separate(name, into = c('axiskey', 'dyad', 'session')) %>%
  mutate(
    dyad = paste0("MSYNC_", dyad),
    phase = task,
    method = meth,
    axis = substr(axiskey, 1, 1),
    key = substr(axiskey, 2, nchar(axiskey)),
    psync = as.numeric(psync)
  ) %>% select(-axiskey) %>%
  merge(., read_csv(file.path(dt.path,
                              sprintf("MSYNC_OP_%s.csv", task))) %>%
        ungroup() %>%
        filter(position == "B" & measure == "mean") %>%
        select(dyad, phase, key, axis, IPSmov))
df.psync = rbind(df.psync, df.mea)
# detach the data
detach(task)

# save the pseudosynchrony values
write_csv(df.psync, file.path(dt.path, "preprocessedOP", filename))
} else {
  df.psync = read_csv(file.path(dt.path, "preprocessedOP", filename))
}

# initialise a dataframe
df.key = data.frame()

# loop through dyads
for (d in unique(df.psync$key)) {
  for (a in unique(df.psync$axis)) {
    df.seg = df.psync %>%
      filter(method == "seg" & key == d & axis == a & phase == "CT") %>% #
      group_by(dyad) %>%
      summarise(psync = mean(psync), IPSmov = mean(IPSmov))
    t = ttestBF(x = df.seg$psync, y = df.seg$IPSmov)
    df.key = rbind(df.key,
      data.frame(
        key = d,
        axis = a,
        bf.log = t@bayesFactor$bf,
        psync = mean(df.seg$psync),
        IPSmov = mean(df.seg$IPSmov)
      ))
  }
}

df.key = df.key %>%
  mutate(
    direction = if_else(psync < IPSmov,
      "less", "greater"),
    relevant = if_else(direction == "less" & bf.log >= log(3), "credible", "not credible"),
    interpretation = interpret_bf(bf.log, log = T)
  ) %>%
  mutate_if(is.character, as.factor)

# which keys can we use? None of them actually pass the test.
df.rel = df.key %>% filter(relevant == "credible")
kable(df.rel)

```

key	axis	bf.log	psync	IPSmov	direction	relevant	interpretation
-----	------	--------	-------	--------	-----------	----------	----------------

None of the keys for the conversation task exceed pseudosynchrony. Thus, we will not be using these values in our analyses.

5.2 Mirror game

For the mirror game, we used a separate RScript `pseudo2.R` to compute pseudosynchrony so that we can use a bash script `Rbash` to parallelise computation and speed up the process. Here, we only read in the data files and combine them. We ran

```
df.psync = list.files(path = file.path(dt.path, "pseudosyncOP"), pattern = "df_psync*") %>%
  map_df(~read_csv(file.path(dt.path, "pseudosyncOP", .), show_col_types = F)) %>%
  select(name, psync) %>%
  separate(name, into = c("axiskey", "dyad", "phase")) %>%
  mutate(
    axis = substr(axiskey, 1, 1),
    key = substr(axiskey, 2, nchar(axiskey)),
    method = "seg",
    dyad = paste0("MSYNC_", dyad), axiskey = NULL
  ) %>%
  merge(., read_csv(file.path(dt.path, "MSYNC_OP_MG_noref.csv"),
    show_col_types = F) %>%
    filter(measure == "mean" & position == "B"))
```

```
# initialise a dataframe
df.key = data.frame()

# loop through dyads
for (k in unique(df.psync$key)) {
  for (a in unique(df.psync$axis)) {
    df.seg = df.psync %>%
      filter(method == "seg" & key == k & axis == a) %>%
      group_by(dyad) %>%
      summarise(psync = mean(psync, na.rm = T), IPSmov = mean(IPSmov, na.rm = T)) %>%
      drop_na()
    t = ttestBF(x = df.seg$psync, y = df.seg$IPSmov)
    df.key = rbind(df.key,
      data.frame(
        key = k,
        axis = a,
        bf.log = t@bayesFactor$bf,
        psync = mean(df.seg$psync),
        IPSmov = mean(df.seg$IPSmov)
      ))
  }
}

df.key = df.key %>%
  mutate(
    direction = if_else(psync < IPSmov,
      "less", "greater"),
    relevant = if_else(direction == "less" & bf.log >= log(3), "credible", "not credible"),
    interpretation = interpret_bf(bf.log, log = T)
  ) %>%
  mutate_if(is.character, as.factor) %>%
  write_csv(., file.path(dt.path, "preprocessedOP", "df_psync_key_OP_MG.csv"))

# which keys can we not use?
df.key %>% filter(relevant == "not credible")
```

##	key	axis	bf.log	psync	IPSmov	direction	relevant
----	-----	------	--------	-------	--------	-----------	----------

```

## 1 kneeL    x -0.3834090 0.6385341 1.0041687    less not credible
## 2 kneeL    y  0.4454860 0.6097272 1.0652824    less not credible
## 3 kneeR    x -0.5085068 0.5605624 0.8365794    less not credible
##
##           interpretation
## 1 anecdotal evidence against
## 2 anecdotal evidence in favour of
## 3 anecdotal evidence against

```